

EduSelect — Complete System Architecture & Engineering Blueprint

A Comprehensive Technical Architecture Guide for Developers, Engineers, and Technical Collaborators.

Project: **EduSelect (College Discovery & Prediction Platform)**

Version: `1.0.0` | Target Ecosystem: Modern Web / Full-Stack Next.js

Authors: EduSelect Engineering Team

Table of Contents

- 1. [Executive Summary & Product Vision](#)
- 2. [High-Level Architecture Diagram](#)
- 3. [Technology Stack Matrix](#)
- 4. [Codebase & Directory Topology](#)
- 5. [Core Architectural Design Patterns](#)
- 6. [Subsystem Deep Dives](#)
 - [6.1 Multi-Facet Search & Discovery Engine](#)
 - [6.2 KCET 2026 Rank & College Predictor Engine](#)
 - [6.3 College Comparison Matrix Engine](#)
 - [6.4 Dynamic Institutional Profile Pages](#)
 - [6.5 28 Indian States & Geographic Hubs](#)
 - [6.6 Admin CMS & Governance Portal](#)
- 7. [Authentication, Authorization & Security](#)
- 8. [Data Architecture & Persistence Layer](#)
- 9. [REST API Specification Matrix](#)
- 10. [Client State & Data Fetching Strategy](#)
- 11. [Performance Optimizations & Production Scaling Roadmap](#)
- 12. [Developer Quick Start & Environment Setup](#)

1. Executive Summary & Product Vision

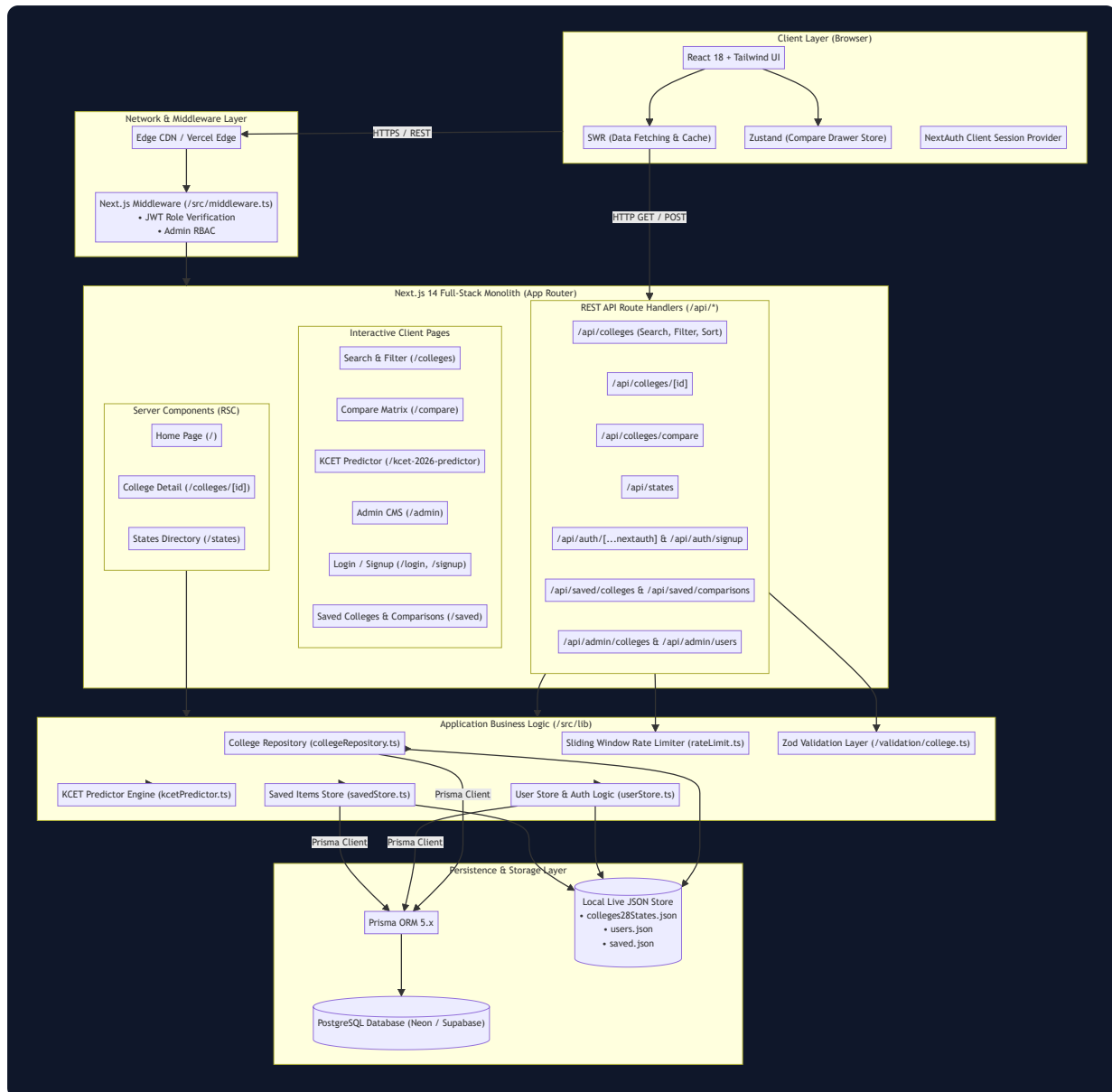
EduSelect is an enterprise-grade full-stack higher education platform built to guide students across India through institution discovery, course evaluation, side-by-side college comparisons, and competitive entrance cut-off predictions.

Key Capabilities:

- **Pan-India Coverage:** Institutional data spanning all 28 Indian States and Union Territories.
 - **Instant Multi-Parametric Search:** Filter by state, city, institution type (`GOVERNMENT` , `PRIVATE` , `DEEMED`), fee ceilings, NIRF ratings, and entrance exams (JEE, KCET, COMEDK, MHT-CET, WBJEE, TNEA, etc.).
 - **KCET 2026 Predictive Scoring:** Mathematical composite rank and college admission predictor based on PCM Board exam scores (50%) and KCET entrance scores (50%), factoring in reservation categories (`GM` , `2A` , `2B` , `3A` , `3B` , `SC` , `ST`) and quotas.
 - **Side-by-Side College Matrix:** Multi-college comparative analytics benchmarking fees, placement records, highest/average packages, and accreditation side-by-side.
 - **Admin CMS:** Internal dashboard enabling administrative teams to update institutional metadata, manage courses, and control user access.
-

2. High-Level Architecture Diagram

EduSelect is architected as a **Next.js 14 App Router Full-Stack Monolith**. Frontend UI components, server-side data loaders, middleware guards, and REST API handlers live within a single repository, sharing TypeScript definitions, validation schemas, and database connectors.



3. Technology Stack Matrix

Layer	Technology	Version	Purpose & Rationale
Framework	Next.js	14.2.15	App Router paradigm for hybrid Server Components (RSC) and Client Components with zero CORS complexity.
Language	TypeScript	5.x	End-to-end type safety spanning DB schemas, API responses, and React components.
UI Library	React	18.3.1	Modern concurrent rendering, Suspense fallbacks, and component-driven architecture.
Styling	TailwindCSS	3.4.1	Utility-first responsive design, dark mode tokens, and custom animation utilities.
State Management	Zustand	4.5.5	Lightweight client state for the persistent 3-college Comparison Drawer across navigation.
Client Fetching	SWR	2.2.5	Stale-While-Revalidate caching, automatic deduplication, and optimistic UI updates.
ORM	Prisma	5.22.0	Declarative relational schema, type-safe query generation, and database migrations.
Database	PostgreSQL	15+	Relational storage hosted on Neon Serverless with foreign key cascade integrity.
Authentication	NextAuth.js	4.24.15	Credentials Provider, stateless JWT token strategy, and secure HTTP-only cookies.
Hashing	bcryptjs	2.4.3	Cryptographic password hashing (10 salt rounds) for user credentials.
Validation	Zod	3.23.8	Single-source schema validation shared between API query parsers and client forms.
Data Export	SheetJS (xlsx)	0.18.5	Exporting side-by-side college comparisons to Excel/CSV worksheets.
Notifications	react-hot-toast	2.6.0	Micro-interaction toast alerts for bookmarks, comparison actions, and errors.

4. Codebase & Directory Topology

```

edu-select/
├─ prisma/
│  └─ schema.prisma      # Declarative PostgreSQL schema & relations
│  └─ seed.ts            # Database seeder populating colleges, courses & placements
├─ public/
│  └─ images/            # Institutional photos, university emblems, state hero graphic
│  └─ favicon.ico        # Platform branding favicon
├─ src/
│  └─ app/               # Next.js 14 App Router (Pages, Layouts & Endpoints)
│     └─ admin/          # Internal Admin CMS Portal
│        └─ colleges/    # Institution editor & course manager
│        └─ login/       # Admin dedicated authentication screen
│        └─ states/      # State metadata & regional college count management
│        └─ users/       # User role assignment & account suspension
│        └─ layout.tsx   # Admin sidebar & breadcrumbs layout
│        └─ page.tsx     # Executive metrics dashboard
│     └─ api/            # REST API Handlers
│        └─ admin/       # Secured admin endpoints (Colleges & Users)
│        └─ auth/        # NextAuth [...nextauth] & User Signup
│        └─ colleges/    # GET /api/colleges & /api/colleges/[id] & /compare
│        └─ saved/       # Saved colleges & custom comparison lists
│        └─ states/      # Aggregated state directory & entrance exam counts
│     └─ colleges/       # College Discovery & Directory
│        └─ [id]/        # College Profile Page (Overview, Courses, Placements, Reviews)
│        └─ page.tsx     # Multi-criteria search & filter directory
│     └─ compare/        # Side-by-side college comparison matrix
│     └─ kcet-2026-predictor/ # KCET Rank & Admission Probability Calculator
│     └─ login/          # User Sign-In screen
│     └─ signup/         # User Registration screen
│     └─ saved/          # User Bookmarks & Saved Comparisons dashboard
│     └─ states/         # 28 States Geographical Directory Hub
│     └─ globals.css     # Tailwind base directives & color variables
│     └─ layout.tsx      # Global Root Layout (Navbar, Toast, CompareBar, Footer)
│     └─ page.tsx        # Interactive Landing Page
│  └─ components/        # Modular UI Component Library
│     └─ features/       # Feature-specific composite components
│        └─ CollegeActionButtons.tsx # Save & Compare interactive triggers
│        └─ CollegeCard.tsx      # Institution discovery card
│        └─ CollegeLogoBadge.tsx # Dynamic fallback avatar badge
│        └─ CompareBar.tsx       # Persistent comparison floating drawer
│        └─ StateCardImage.tsx   # Regional landscape banner
│     └─ providers/          # Context providers (NextAuth SessionProvider)
│     └─ ui/                # Atomic Design System (Button, Badge, Card, Input, Modal, Tab)
│  └─ data/                 # Hybrid Data Store Files
│     └─ colleges28States.json # Master repository of Indian colleges (~1.1 MB)
│     └─ saved.json          # User saved items snapshot
│     └─ users.json          # Local accounts snapshot with hashed passwords
│  └─ lib/                  # Business Logic, Repositories & Utilities
│     └─ auth.ts             # NextAuth configuration & Credentials provider setup
│     └─ collegeRepository.ts # Hybrid Data Access Layer (JSON + Prisma dual read/write)
│     └─ constants.ts        # Global app constants, state lists & filter defaults
│     └─ db.ts               # Singleton Prisma client instance
│     └─ kcetPredictor.ts    # KCET Composite Rank & Cutoff Calculation Engine
│     └─ rateLimit.ts        # In-memory sliding window IP rate limiter
│     └─ savedStore.ts        # Bookmark & Comparison persistence layer
│     └─ stateConfig.ts      # 28 States metadata, slugs, icons & gradient styles
│     └─ userStore.ts         # User management & authentication storage
│     └─ store/              # Zustand state stores (useCompareStore)
│     └─ validation/         # Zod input schemas (collegeQuerySchema)

```

```
| ├── middleware.ts      # Route protection & Admin RBAC verification
| ├── types/            # Global TypeScript interfaces & DTOs
| ├── package.json      # Project dependencies & scripts
| ├── tailwind.config.ts # Tailwind theme tokens & design system
| └── tsconfig.json     # TypeScript compiler configuration
```

5. Core Architectural Design Patterns

1. Monolithic Next.js 14 App Router

- **Single Deployable Unit:** Eliminates the deployment overhead of separate backend microservices.
- **Zero CORS Configuration:** Frontend and API routes share the exact same host origin.
- **Shared Type Definitions:** Data Transfer Objects (DTOs), Zod schemas, and Prisma types are shared directly without build-step code generation.

2. Hybrid Data Access Pattern (Resilience Architecture)

To guarantee high availability and enable seamless local development without mandatory cloud database dependencies:

1. **Primary Database:** PostgreSQL connected through Prisma ORM for relational queries, foreign key cascading, and production workloads.
2. **Local Live JSON Repository:** `src/data/colleges28States.json` serves as an offline-first, high-performance data store.
3. **Dual-Write Synchronization:** When an administrator edits college information via `/api/admin/colleges`, the repository updates the live JSON file and simultaneously syncs with PostgreSQL if a live database connection exists.

3. Separation of Client and Server Boundaries

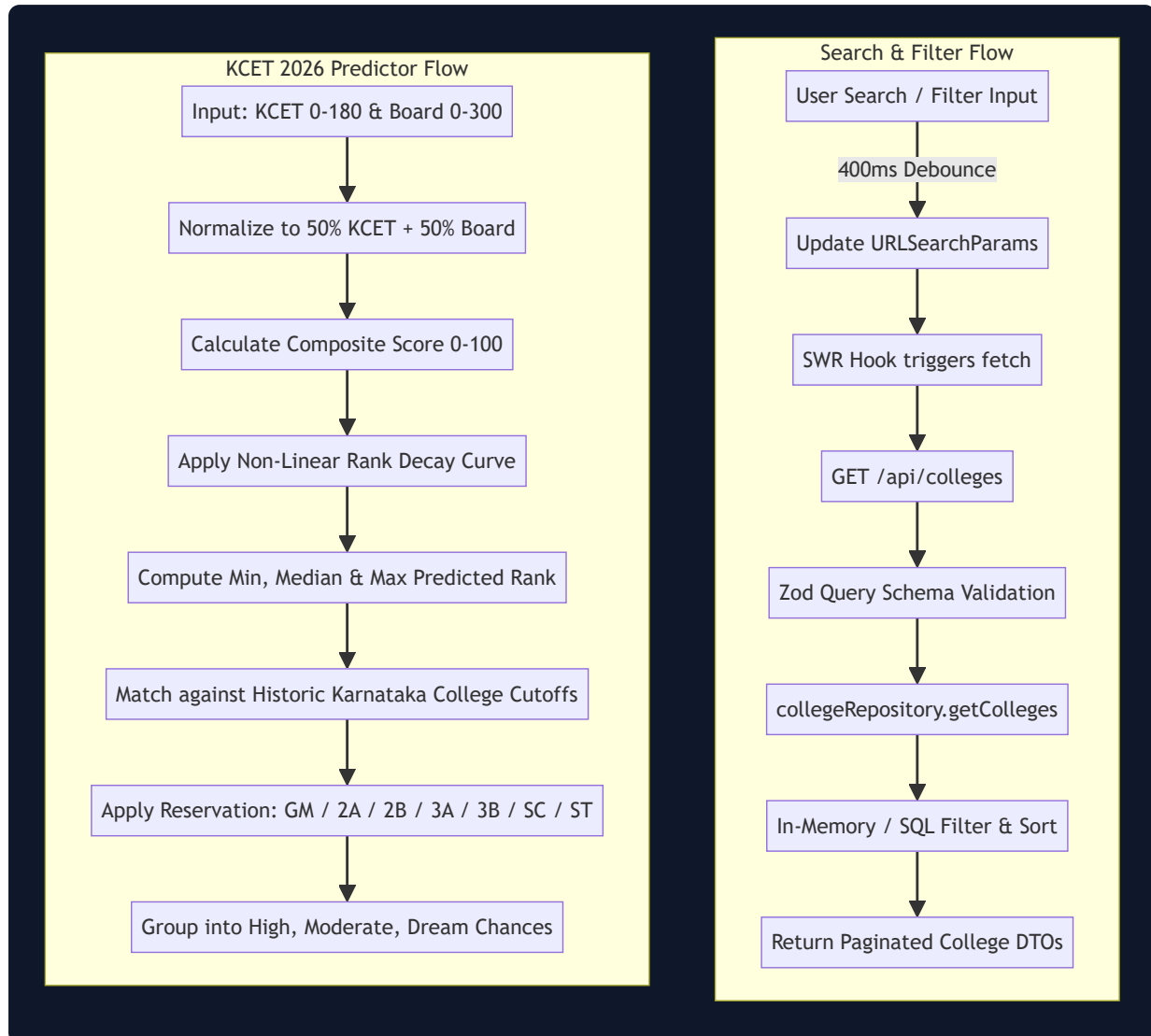
- **Server Components (RSC):** Render the static skeleton, college profile metadata, and state hero graphics without sending JavaScript to the browser.
- **Client Components ('use client'):** Reserved for interactive micro-components, including debounced search bars, slider filters, the comparison tray, and tab switchers.

4. Single Source of Truth for Search State (URL Query Params)

All filtering criteria (search keyword, state, city, type, fees, rating, page) are serialized into the browser URL (`/colleges?search=RVCE&type=GOVERNMENT&minRating=4`). This architecture ensures:

- Filtered results are instantly **bookmarkable and shareable**.
 - Browser **Back** and **Forward** buttons work naturally.
 - State is restored immediately upon page reload.
-

6. Subsystem Deep Dives



6.1 Multi-Facet Search & Discovery Engine

- **Debounced Text Search (400ms):** Prevents network saturation while the user types. Queries match college names, cities, states, affiliations, overview texts, entrance exam codes, and individual course branch codes.
- **Multi-Factor Filtering:**
 - **Type:** Enums `GOVERNMENT` , `PRIVATE` , `DEEMED` .
 - **State & City:** Exact case-insensitive matching across 28 states.
 - **Fee Range:** Lower and upper bound numerical comparisons.
 - **Rating Threshold:** Minimum user rating filter (0.0 to 5.0).
- **Pagination:** Offset-based pagination with customizable `limit` (default: 12, max: 500) and page index calculations.

6.2 KCET 2026 Rank & College Predictor Engine

The predictor located in `src/lib/kcetPredictor.ts` implements Karnataka Examination Authority's (KEA) official engineering rank computation guidelines:

$$\text{KCET Percentage} = \frac{\text{Physics} + \text{Chemistry} + \text{Maths}}{180} \times 100$$

$$\text{Board Percentage} = \frac{\text{Board Physics} + \text{Board Chemistry} + \text{Board Maths}}{300} \times 100$$

$$\text{Composite Score} = (0.50 \times \text{KCET Percentage}) + (0.50 \times \text{Board Percentage})$$

- **Non-Linear Rank Estimation Curve:** Accounts for score clustering at higher score intervals ($>90\%$) and higher density in median score ranges ($60\% - 75\%$).
- **Quota & Category Matrix:** Dynamically selects the appropriate cutoff threshold for the candidate's category (GM, 2A, 2B, 3A, 3B, SC, ST) and applies reservation adjustments for Rural and Kannada Medium quotas.
- **Admission Probability Classification:**
 - **High Chance:** Candidate's predicted rank is comfortably within the historical cutoff ($\text{Rank} \leq \text{Cutoff} \times 0.85$).
 - **Moderate Chance:** Candidate's predicted rank is close to the cutoff ($\text{Cutoff} \times 0.85 < \text{Rank} \leq \text{Cutoff} \times 1.10$).
 - **Dream / Ambitious:** Candidate's predicted rank is slightly above the historical cutoff ($\text{Cutoff} \times 1.10 < \text{Rank} \leq \text{Cutoff} \times 1.35$).

6.3 College Comparison Matrix Engine

- **Global Comparison State (`useCompareStore`):** Zustand-powered client store supporting up to 3 simultaneously selected institutions.
- **Persistent Bottom Drawer (`CompareBar.tsx`):** Displays selected institutions with thumbnail badges, quick removal triggers, and a direct CTA to `/compare`.
- **Comparative Metrics Table:** Compares Annual Tuition Fees, NIRF/User Rating, State & City, Campus Type, 3-Year Placement Packages (Highest & Average), Branch Offerings, and Regulatory Approvals (AICTE, NBA, NAAC).
- **Export Utility:** Integrated Excel workbook generation (`xlsx`) allowing students to download comparison sheets.

6.4 Dynamic Institutional Profile Pages (`/colleges/[id]`)

- **Dual Identifier Resolution:** Fetches institution profiles seamlessly by either unique CUID/UUID or human-readable URL slug (e.g., `/colleges/rv-college-of-engineering-bangalore`).
- **Tabbed Information Architecture:**
 1. **Overview:** Institutional history, campus infrastructure, accreditations, and affiliation details.
 2. **Courses & Fees:** Comprehensive branch tables with duration, annual fee breakdown, and seat intake.
 3. **Placements:** 3-year historical trends showing Average Package, Highest Package, Placement %, and list of Top Recruiters (Google, Amazon, Microsoft, TCS, etc.).
 4. **Reviews & Ratings:** Student testimonials, verified ratings, and campus feedback.

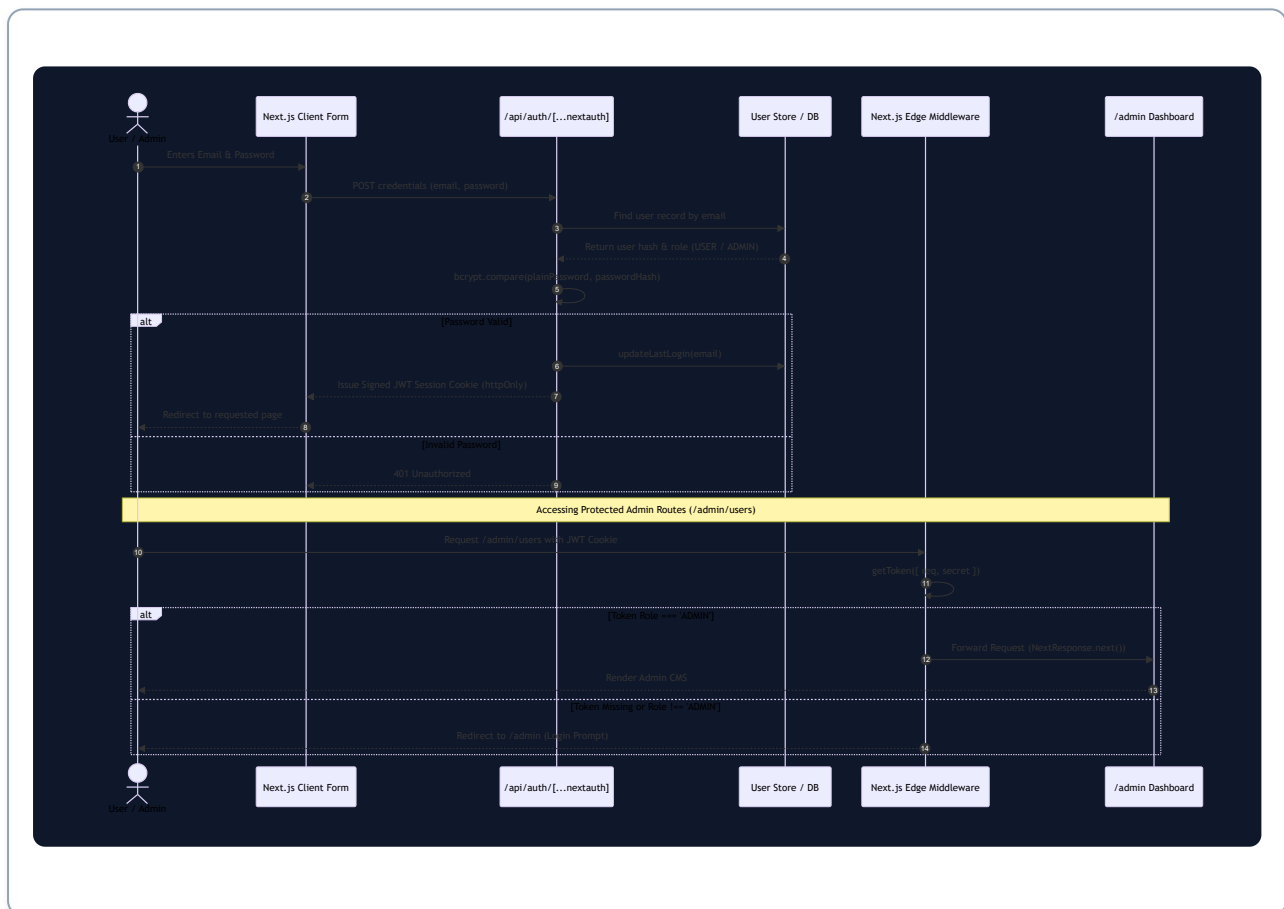
6.5 28 Indian States & Geographic Hubs (/states)

- Provides dedicated regional exploration hubs for all 28 Indian States.
- Incorporates custom state visual styling: localized emoji identifiers, curated color gradients, and state landscape imagery.
- Automatically calculates real-time college counts and filters by regional entrance exams (e.g., KCET/COMEDK in Karnataka, MHT-CET in Maharashtra, WBJEE in West Bengal, TNEA in Tamil Nadu).

6.6 Admin CMS & Governance Portal (/admin)

- **Executive Analytics:** High-level counts of total registered institutions, user accounts, and active comparisons.
- **Institutional Management:** Full editing capabilities for college names, tuition fees, ratings, overview text, and course configurations.
- **User Governance:** View registered users, examine account creation dates, and toggle account statuses between **ACTIVE** and **SUSPENDED**.

7. Authentication, Authorization & Security



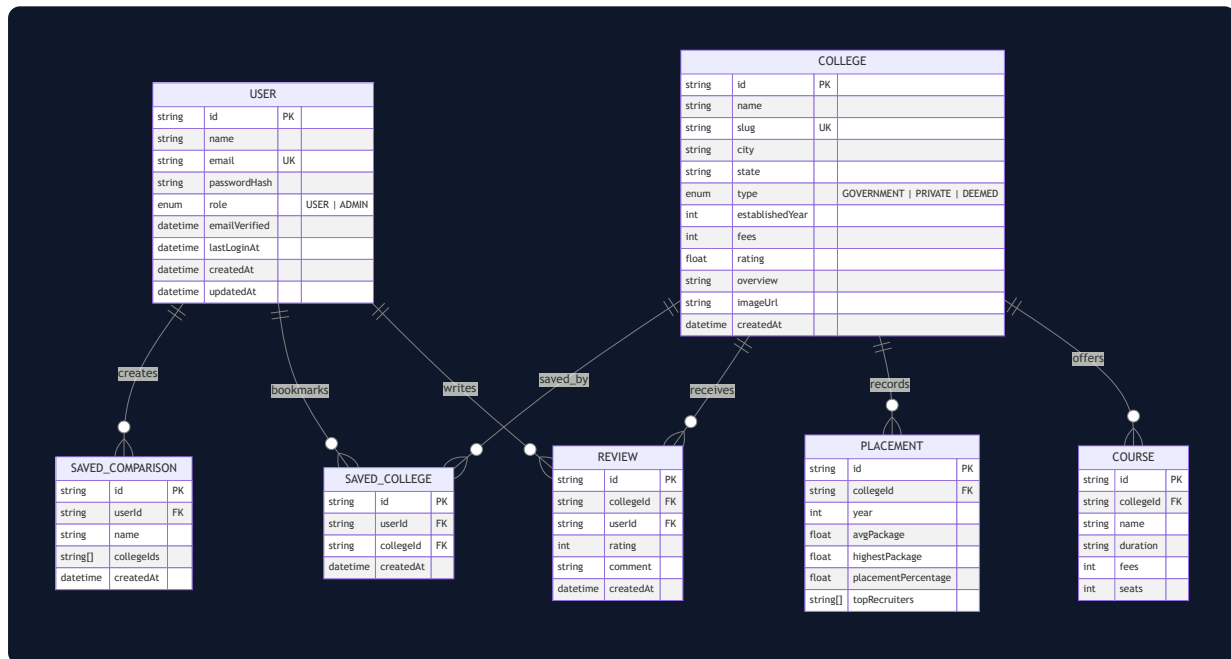
Security Defenses:

1. **Stateless JWT Sessions:** Eliminates database lookups on every route navigation, minimizing latency while maintaining session validity for 30 days.

- 2. Cryptographic Password Security:** Enforces `bcryptjs` with salt round factor 10. Passwords are never stored in plaintext.
- 3. Route Protection Middleware:** Intercepts incoming requests at the Edge (`src/middleware.ts`). Restricts administrative routes (`/admin/users/*` , `/admin/settings/*`) exclusively to authenticated users with `role: 'ADMIN'` .
- 4. Input Validation & Sanitization:** All incoming REST queries are parsed and type-coerced through Zod schemas to eliminate malformed inputs and parameter tampering.
- 5. Sliding Window Rate Limiter:** An in-memory sliding window limiter in `src/lib/rateLimit.ts` safeguards endpoints against brute-force attacks and volumetric request abuse.

8. Data Architecture & Persistence Layer

8.1 Entity-Relationship (ER) Diagram



8.2 Database Indexing Strategy

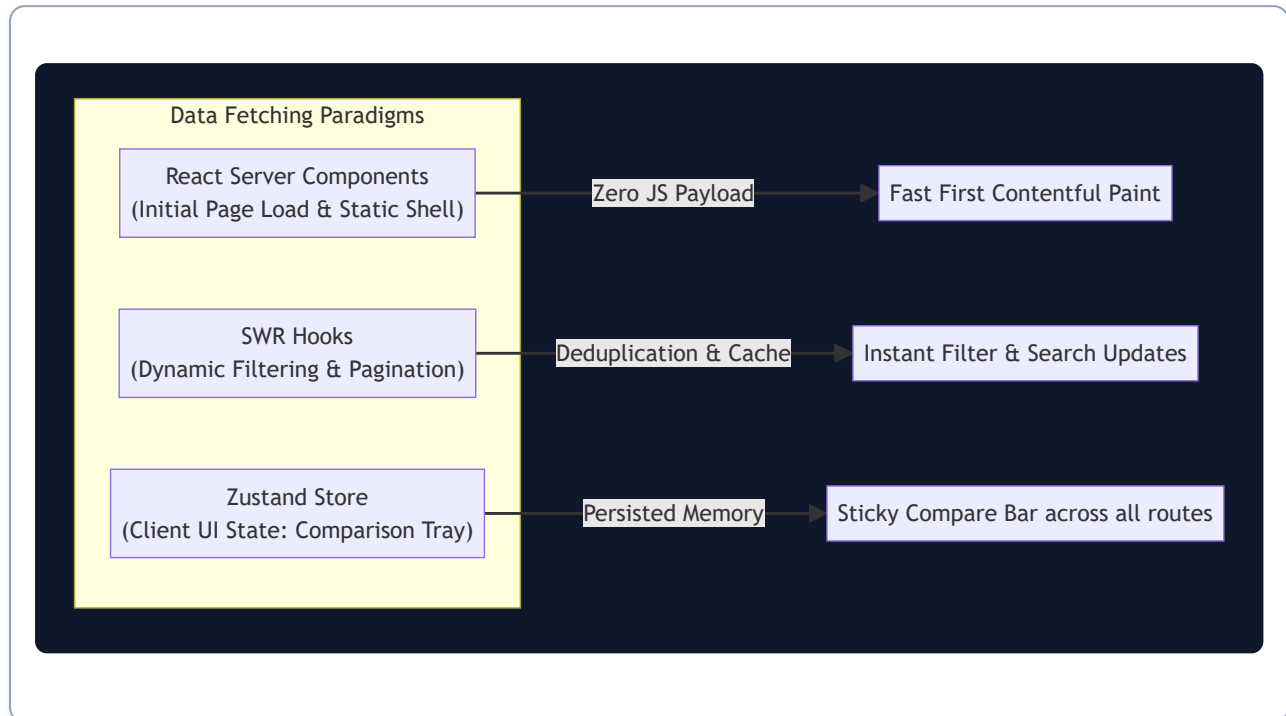
To maintain sub-50ms query times at scale, the Prisma schema defines targeted database indexes:

- User** : `@@index([email])` , `@@index([role])`
- College** : `@@index([city])` , `@@index([state])` , `@@index([fees])` , `@@index([rating])`
- SavedCollege** : `@@unique([userId, collegeId])` (Prevents duplicate bookmarks at the database level)

9. REST API Specification Matrix

Method	Endpoint	Access Level	Description	Key Request / Query Parameters
GET	/api/colleges	Public	Paginated list of colleges matching search & multi-filter criteria	search, city, state, type, minFees, maxFees, minRating, sortBy, sortOrder, page, limit
GET	/api/colleges/[id]	Public	Fetch single college profile by ID or URL slug	Path param: id (e.g. rvce or CUID)
POST	/api/colleges/compare	Public	Retrieve multiple full college records for comparison matrix	Body: { ids: string[] }
GET	/api/states	Public	Summary list of 28 states with college counts and exam associations	None
POST	/api/auth/signup	Public	Register a new student user account	Body: { name, email, password }
POST	/api/auth/[...nextauth]	Public	NextAuth handler for user login, token refresh, and sign-out	Body: { email, password }
GET	/api/saved/colleges	Authenticated	Retrieve all bookmarked colleges for the logged-in user	Session Cookie
POST	/api/saved/colleges	Authenticated	Bookmark or remove a college from user's saved list	Body: { collegeId: string }
GET	/api/saved/comparisons	Authenticated	Retrieve all custom saved comparisons for current user	Session Cookie
POST	/api/saved/comparisons	Authenticated	Save a new comparison group	Body: { name: string, collegeIds: string[] }
GET	/api/admin/colleges	Admin Only	Full college catalog for management console	Session Cookie (role: ADMIN)
PUT	/api/admin/colleges	Admin Only	Update institutional details, fees, rating, or courses	Body: { id, updates: Partial<CollegeDetail> }
GET	/api/admin/users	Admin Only	List registered users with account status and roles	Session Cookie (role: ADMIN)
PUT	/api/admin/users	Admin Only	Update user role or toggle suspension status	Body: { userId, status, role }

10. Client State & Data Fetching Strategy



1. React Server Components (RSC):

- Implemented on landing pages and detail profile views (`/colleges/[id]`).
- Server-renders critical HTML on the initial hit, dramatically accelerating First Contentful Paint (FCP) and boosting search engine SEO indexing.

2. Client-Side SWR Fetching:

- Powers the search page (`/colleges`) and dynamic widgets.
- Provides automatic cache deduplication, background revalidation, and built-in loading states (`isLoading` , `error`).

3. Zustand Comparison Store (`src/lib/store/useCompareStore.ts`):

- Manages the selected institution comparison IDs (`string[]` , maximum 3).
- Survives route transitions across the entire site without requiring a heavyweight React Context Provider tree.

11. Performance Optimizations & Production Scaling Roadmap

Current Engineering Optimizations:

- **Zero Layout Shift (CLS):** Pre-allocated skeleton loaders (`Skeleton.tsx`) while SWR loads data.
- **Image Optimization:** Responsive WebP image rendering with fallback SVG logo badges (`CollegeLogoBadge.tsx`).
- **Debounced Network Requests:** 400ms delay on all keystroke inputs to protect API endpoints from excessive load.

Production Scaling Roadmap:

1. Cursor-Based Keyset Pagination:

- *Current:* Offset-based pagination (`OFFSET N LIMIT M`).
- *Roadmap:* Transition to Keyset/Cursor pagination (`WHERE id > :cursor LIMIT 12`) when database size exceeds 100,000+ institutional records.

2. Distributed Redis Caching:

- *Current:* In-memory sliding window rate limiter and JSON fallback.
- *Roadmap:* Deploy Upstash Redis for distributed cache storage and multi-instance rate limiting across Vercel Serverless Regions.

3. Third-Party Object Storage:

- *Current:* Static images served from `/public/images/` .
- *Roadmap:* Integrate AWS S3 or Cloudinary with an automated image upload pipeline for verified campus imagery.

4. Social OAuth Providers:

- Extend NextAuth configuration to support Google and LinkedIn OAuth single sign-on (SSO).

12. Developer Quick Start & Environment Setup

1. Prerequisites

- **Node.js:** `v18.17.0` or higher
- **npm:** `v9.x` or higher
- **PostgreSQL:** Local PostgreSQL instance or a free serverless database on [Neon.tech](#)

2. Environment Configuration

Create a `.env` file in the project root:

```
# Database Connection (Neon Serverless or Local Postgres)
DATABASE_URL="postgresql://username:password@ep-cool-project.ap-southeast-1.aws.neon.tech/neond
# NextAuth Authentication Secret (Minimum 32 random characters)
NEXTAUTH_SECRET="e93b2a8d4f0c7e6b1a5d8f2c3e4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2"
NEXTAUTH_URL="http://localhost:3000"
```

3. Dependency Installation & Database Initialization

```
# 1. Install dependencies
npm install

# 2. Generate Prisma Client
npx prisma generate

# 3. Push schema to database
npx prisma db push

# 4. (Optional) Run Database Seeder
npm run build # or npx prisma db seed
```

4. Running the Development Server

```
npm run dev
```

Open <http://localhost:3000> in your browser.

5. Pre-Seeded Default Accounts

Account Type	Email Address	Password	Permissions
Administrator	admin@collegediscovery.com	Admin@123456	Full Admin CMS Access (/admin)
Student User	aarav.sharma@example.com	password123	Bookmark, Compare, Predict (/saved)

This document is maintained as the authoritative architectural blueprint for the EduSelect codebase.